SymPy Bug Report

1 Bug 35: solveset includes negative real non-solutions for a reciprocal square-root equation

1.1 Minimal Reproducer

```
from sympy import Eq, S, sqrt, solveset, symbols

x = symbols("x")
lhs = sqrt(x) * sqrt(1/x)
sol = solveset(Eq(lhs, 1), x, domain=S.Complexes)

print("solution:", sol)
print("contains -1:", sol.contains(S.NegativeOne))
print("lhs at -1:", lhs.subs(x, -1))
print("rhs:", S.One)
print("residual at -1:", (lhs - 1).subs(x, -1))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solution: Complement(Complexes, {0})
contains -1: True
lhs at -1: -1
rhs: 1
residual at -1: -2
```

SymPy reports that every nonzero complex number satisfies $\sqrt{x}\sqrt{1/x} = 1$. In particular, the returned set contains $x = -1$, even though direct substitution gives $\sqrt{-1}\sqrt{-1} = -1$, not 1. Thus the returned set contains a concrete value for which the original equation is false.

1.3 Expected Behavior

The solution set over the principal complex square root should exclude 0 and the negative real axis. Equivalently, -1 and the other negative real instances below must not be included. A conservative solver result would also be acceptable if it preserved the original radical condition instead of returning the unchecked superset $\mathbb{C} \setminus \{0\}$.

1.4 Mathematical Explanation

SymPy uses the principal complex square root. Principal square roots are not multiplicative across the negative real branch cut, so

$$\sqrt{x}\sqrt{1/x} = \sqrt{x(1/x)}$$

is not a valid identity for arbitrary complex x. At the representative counterexample,

$$\sqrt{-1}\sqrt{1/(-1)} = \sqrt{-1}\sqrt{-1} = i \cdot i = -1 \neq 1.$$

More generally, write $x = re^{i\theta}$ using the principal argument $\theta \in (-\pi, \pi]$. For $\theta \in (-\pi, \pi)$, the reciprocal has argument $-\theta$, and the product of the principal square roots is 1. For a negative real x, however, $\theta = \pi$ and $1/x$ is again negative real, so both principal square roots have argument $\pi/2$, and their product has argument π. The product is therefore -1, not 1. The observed answer is not a harmless alternate representation: it includes values that make the original equation false.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant source file as `sympy/solvers/solveset.py`. The affected function is `_solve_radical`, including its local candidate-checking logic. The traced call path is: `solveset` rewrites the equation as $\sqrt{x}\sqrt{1/x}-1$; the direct inverter cannot solve the radical product; `_solveset` calls `unrad`; and then `_solve_radical` processes the radical-free result.

For this input, `unrad` applied to $\sqrt{x}\sqrt{1/x} - 1$ returns the superset equation `(0, [])`, because squaring the radical product gives $x(1/x)-1 = 0$. After denominator removal, this becomes $\mathbb{C}\setminus\{0\}$. The root-cause note locates the responsible fallback in `_solve_radical`: finite candidate sets are checked against the original equation, but broad non-finite sets are returned unchanged.

```
else:
    # XXX: There should be more cases checked here.
    return solutions
```

This behavior is unsafe for radical elimination because `unrad` can produce a superset of the true solution set. In this case, the superset is valid only after adding the missing principal-branch restriction. Since the infinite candidate set is not validated against the original radical equation, negative real non-solutions such as -1 remain in the final result. The supported fix direction is to avoid returning infinite radical-removal solution sets as final answers unless the original equation has been enforced; a conservative result could be a `ConditionSet` over the candidate set, while a more complete fix would derive and subtract the principal-branch failure set.

1.6 Additional Failing Instances

The independent verification checks the representative value and five additional negative real values. In each case, SymPy's returned $\mathbb{C}\setminus\{0\}$ contains the value, while direct substitution into the original equation gives residual -2. The same reasoning applies across the family: for every negative real x, both x and $1/x$ lie on the principal square-root branch cut, so the product of the two principal square roots is -1, not 1.

Input / Expression	SymPy behavior	Expected behavior
$x = -1$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded
$x = -2$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded
$x = -3$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded
$x = -\frac{1}{2}$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded
$x = -5$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded
$x = -\frac{7}{3}$, $\sqrt{x}\sqrt{1/x} = -1$	Included; residual -2	Excluded

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material documents the same mathematical family: SymPy's square-root documentation discusses principal-square-root behavior, SymPy's solveset documentation describes returning the set where an equation is true, and public math discussions cover failures of identities such as $\sqrt{xy} = \sqrt{x}\sqrt{y}$ across branch cuts. The artifact found no public issue, pull request, release note, Stack Overflow report, or mailing-list discussion for this exact `solveset` equation returning $\mathbb{C} \setminus \{0\}$. The deduplication verdict is therefore a new instance of a known failure mode: the general branch-cut issue is understood, but this exact reproducible solver result is previously unreported and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```python
import pytest

from sympy import Eq, Rational, S, simplify, sqrt, solveset, symbols


@pytest.mark.parametrize(
    "bad_value",
    [
        S.NegativeOne,
        S(-2),
        S(-3),
        Rational(-1, 2),
        S(-5),
        Rational(-7, 3),
    ],
)
def test_solveset_sqrt_x_sqrt_reciprocal_excludes_negative_real_axis(bad_value):
    x = symbols("x")
    lhs = sqrt(x) * sqrt(1/x)

    sol = solveset(Eq(lhs, 1), x, domain=S.Complexes)

    assert sol.contains(bad_value) is S.false
    assert simplify((lhs - 1).subs(x, bad_value)) != 0
```